Reflection Report — ShopEasy Term Project

This short reflection summarizes what I learned while building the ShopEasy test suite, which techniques revealed interesting behaviors in the starter code, which techniques gave the best return on effort, how the test suite fits the testing pyramid, and what I would test next with one more week.

Which bugs or surprising behaviors did each technique reveal?

Specification-based tests (Task 1) for `PriceCalculator` were useful for making implicit expectations explicit. By exercising zero, typical, boundary and invalid inputs I observed that the provided formula will happily accept negative discounts (treating them like surcharges) and discount rates greater than 100%, which can produce negative prices. Those behaviors are not necessarily "bugs" in the arithmetic sense, but they are surprising from an API-design standpoint because callers would usually expect pre-conditions such as `base >= 0` and `0 <= discount <= 100`. To make the contract explicit I added assert preconditions to `PriceCalculator.calculate` so invalid inputs fail fast when assertions are enabled.

Structural tests (Task 2) for `ShoppingCart` revealed the expected branches in normal cart management logic. Tests confirmed that `addItem` merges quantities when the product is already present, that `removeItem` is a no-op when the product is missing, and that `updateQuantity` enforces a positive quantity by throwing `IllegalArgumentException`. I also added tests that exercise `getItems()` immutability (attempting a modifying operation raises `UnsupportedOperationException`) and `applyDiscount` behaviour. Running JaCoCo (or inspecting the coverage report) was the guide to find which branches remained untested and to add focused cases to reach the branch-coverage target.

Design-by-contract (Task 3) is a lightweight, high-value addition: adding `assert` statements to the production code (in `ShoppingCart`, `PriceCalculator`, and `OrderProcessor`) makes pre-conditions, post-conditions and invariants explicit at runtime when tests run with `-ea`. This caught surprising or undesired inputs during development and made failures easier to diagnose. Note: to validate the contracts we must run tests with assertions enabled.

Property-based testing (Task 4) gave a complementary view by checking invariants across many automatically generated inputs. I wrote properties for non-negativity, identity (0% discount and tax returns the base price), monotonicity of discount, and cart commutativity. These properties are effective at catching subtle corner cases and reasoning about general behavior rather than point examples. They also hinted at the requirement that discounts should be bounded, which we enforce via contract assertions.

Mocks and stubs (Task 5) with Mockito allowed isolated testing of `OrderProcessor`. Using mocks for `InventoryService` and `PaymentGateway` I verified the happy path (inventory OK and payment succeeds), inventory failures, payment failures, and the partial-availability case. These tests validate the orchestration logic and inter-component contracts without wiring up real dependencies.

Which technique was most effective per unit of effort?

For this project the most effective single approach was focused unit testing combined with lightweight mocking. A small number of well-chosen unit tests plus a few mock-driven integration points exposed the majority of behavior issues: interaction mistakes, incorrect error handling, and boundary handling. Structural testing guided by coverage is also high-value but takes more effort to instrument and iterate on. Property-based tests add excellent return on investment for invariant checks, but they complement rather than replace conventional unit tests. Design-by-contract assertions are cheap to add and pay off greatly during test runs since they make implicit assumptions explicit.

How the test suite fits the testing pyramid and what is missing

The current suite is heavily weighted toward unit and property tests. That is appropriate for a library-level project, but integration and end-to-end tests are missing. In particular, tests that exercise `OrderProcessor` with a realistic `InventoryService` and `PaymentGateway` implementation would give higher confidence that components work together. Load/concurrency tests for `ShoppingCart` would also be useful if the cart were used concurrently in a multi-threaded environment.

If I had one more week, what I would test next and why

With an additional week I would run PIT mutation testing and fix surviving mutants to harden the test suite, add integration tests that wire the processor to simple, deterministic implementations of `InventoryService` and `PaymentGateway`, expand property-based tests (more providers and edge cases), and add concurrency and fuzz tests for `ShoppingCart` to detect state-related bugs. Mutation testing in particular gives a clear, prioritized list of missing assertions or behaviors worth covering.